

HavenRae Project Management Platform

Website / System Plan

Prepared by Apex Unified (AU) — DCIT 208 Software Engineering, 2026 Session

Client: HavenRae Studios | Contact: Angel Sharon, Operations Manager

Team Name	Apex Unified (AU)
Project Title	HavenRae Project Management Platform
Client	HavenRae Studios
Repository	github.com/RachelNhyira23/DCIT-208-GROUP-25

1. Project Overview

HavenRae Studios is an interior design company specializing in furnishing and styling residential rental units. As the company has grown, coordinating multiple projects across schedules, budgets, supplier details, and task tracking manually has become increasingly difficult, leading to fragmented information and reduced productivity.

This project delivers a centralized project management platform for HavenRae Studios, streamlining project scheduling, task management, budget tracking, supplier management, and project documentation — giving the company better visibility, less administrative overhead, and a system that supports continued growth.

1.1 Client & Stakeholder

- Client organization: HavenRae Studios
- Primary contact: Angel Sharon, Operations Manager

1.2 Team

Member	Role
Jean Afiba Garibah	Product / Client Manager
Kwadwo Adusei Frimpong	Architecture and Context Lead
Adjepong Jesse Ofosu	QA / Validation Lead
Rachel Nhyira Anane-Frimpong	DevOps / Release Lead
Anafo Iman Yelba	UX / Research Lead
Acquaye Daryl Nii Addo	Implementation Lead
Aaron Allotey	Documentation / Evidence Lead

2. User Groups

- **Operations Manager**
 - Oversees projects, schedules, deadlines, and resources.
- **Interior Designers**
 - View project requirements, update progress, upload design documents, and track tasks.
- **Procurement / Sourcing Personnel**
 - Manage furniture, décor purchases, suppliers, and delivery schedules.
- **Project Assistants**
 - Update task status, record project information, and assist with coordination.
- **Management / Business Owner**

- Monitor project progress, budgets, performance metrics, and reports.
- **Clients (optional view)**
 - View project updates, timelines, and approved designs.

3. Core Features

- Project Management — Create, view, update, and track interior design projects from planning to completion.
- Task Management — Assign and track individual tasks within a project, with status updates (To Do, In Progress, Done).
- Budget Tracking — Record estimated and actual budgets per project; flag overspending.
- Supplier & Procurement Management — Track furniture/décor suppliers, purchase orders, and delivery schedules.
- Project Documentation — Upload and organize design documents, contracts, and reference files per project.
- User Roles & Access Control — Different permissions for Operations Managers, Designers, Procurement staff, and Assistants.
- Dashboard & Reporting — At-a-glance view of active projects, upcoming deadlines, and budget status.

3.1 What Will Not Be Built This Semester

- Native mobile applications (the platform will be a responsive web app only)
- Automated supplier invoicing/payment processing
- Client-facing portal beyond basic read-only project status (full client accounts deferred)
- Advanced analytics/forecasting beyond basic budget and progress reporting

4. System Architecture

The platform follows a standard three-tier web architecture: a browser-based frontend, an API backend handling business logic, and a database for persistent storage. The system is accessed over HTTPS, with authentication required for all project-related actions.

4.1 High-Level Components

- Frontend (Web App): the interface used by Operations Managers, Designers, and Procurement staff to create and manage projects.
- Backend / API Server: handles authentication, validation, and business logic; exposes endpoints such as POST /api/projects.
- Database: stores projects, tasks, users, suppliers, and budget records.
- Authentication Service: verifies user identity and role before granting access to protected actions.

4.2 Example Request Flow — “Create a New Project”

This flow illustrates how the system handles one of its core actions, from user click to stored result:

- User (Operations Manager) opens the “New Project” form in the browser.
- Browser resolves the app's domain via DNS and connects over HTTPS.
- Form submission sends a POST request to /api/projects with project name, client, budget, dates, and assigned designer, plus an auth token.
- Server validates required fields, checks the budget is a valid number, and confirms the user's role permits project creation.
- If valid, a new record is inserted into the projects table/collection with a generated ID.
- Server responds with a JSON success message and the new project ID; the frontend redirects to the new project's dashboard.
- If validation fails, the user sees an inline error without losing their form data; if the database is unreachable, no partial record is saved.

5. Recommended Technology Stack

The stack below pairs a React frontend with a Django REST Framework backend. Django was chosen for the backend specifically for its built-in admin panel, ORM, and authentication system, which reduce the amount of boilerplate the team needs to write for a structured, data-heavy system like project and budget tracking.

5.1 Frontend

- **React (with Vite as the build tool) — component-based UI, huge community support, fast dev server.**
- Tailwind CSS — utility-first styling that speeds up building a clean dashboard/forms UI without writing custom CSS from scratch.
- React Router — for navigating between pages (dashboard, project detail, new project form, etc.).

5.2 Backend

- **Django REST Framework (Python) — a mature, batteries-included framework well-suited to a structured system like HavenRae's, with a built-in admin panel, ORM, and authentication system that reduces boilerplate.**
- djangorestframework-simplejwt — for JWT-based authentication, supporting role-based access control (Operations Manager, Designer, Procurement, etc.).
- Django's built-in permissions system — used alongside custom role checks to enforce who can create, view, or edit projects, tasks, and budgets.

5.3 Database

- **PostgreSQL — a relational database, chosen over NoSQL because projects, tasks, budgets, and suppliers have clear structured relationships (a project has many tasks, a budget belongs to a project, etc.), and relational integrity matters for financial data.**
- Django ORM — Django's built-in ORM handles models, queries, and migrations natively, so no separate ORM library (e.g. Prisma) is needed on the backend.

5.4 File Storage

- Cloudinary (free tier) — for storing uploaded design documents/images per project, rather than storing large files directly in the database.

5.5 Hosting & Deployment

- Frontend: Vercel — free tier, auto-deploys from GitHub on every push to main.
- Backend + Database: Render — free tier supports both a Django/Python web service and a managed PostgreSQL instance, simplifying deployment for a student team.

5.6 CI/CD & Testing

- GitHub Actions — runs lint and test checks automatically on every pull request, satisfying the D4 CI/CD baseline requirement.

- Django's built-in test framework (or pytest-django) — for backend unit/integration tests.
- React Testing Library — for frontend component tests.

5.7 Why This Stack Fits HavenRae's Needs

- Django REST Framework's built-in admin panel gives the team (and even HavenRae's Operations Manager) a ready-made interface for inspecting and managing data early, before the custom frontend is fully built.
- Django's “batteries-included” design (built-in ORM, auth, permissions) reduces boilerplate, which helps a 7-person team with mixed experience levels move faster without each person reinventing core plumbing.
- Free-tier hosting (Vercel + Render) keeps deployment costs at zero for an academic project, while still being production-realistic.
- A relational database fits HavenRae's actual data — projects, budgets, tasks, and suppliers are naturally linked records, not loosely structured documents — and Django's ORM is built specifically around this relational model.
- Splitting frontend (React) and backend (Django) does mean the team works across two languages (JavaScript and Python); this is a reasonable trade-off given Django's strength for structured, data-heavy admin-style systems like project management, but it's worth the team confirming everyone is comfortable picking up Python basics early.

6. Non-Functional Requirements

- **Security:** HTTPS for all traffic; role-based access control; no client/budget data exposed to unauthenticated requests.
- **Privacy:** client names, addresses, and budgets are sensitive business data and must not be logged in plaintext or pasted into public AI tools.
- **Reliability:** project creation must fully succeed or fully fail — no partial/corrupted records.
- **Usability:** forms should give clear inline validation messages; dashboard should give an at-a-glance project overview.
- **Maintainability:** code organized through GitHub Issues, feature branches, and reviewed pull requests, with consistent commit and naming conventions.

7. Key Risks & Mitigations

Risk	Description	Mitigation
Client availability	HavenRae Studios may become slow to respond.	Fallback to a secondary client in interior design, events, or small business with similar needs.

Technical risk	Team inexperience with chosen stack.	Early architecture spikes and incremental feature delivery.
Team participation risk	Uneven contribution across 7 members.	Clear individual ownership, GitHub issue tracking, and escalation process per the Team Operating Agreement.
AI misuse risk	Over-reliance on AI-generated content without understanding.	AI usage logged in AI_USAGE.md; every member must be able to explain their own work.

8. Semester Delivery Timeline

ID	Deadline	Deliverable
D0	24 Jun 2026	Team Charter, Client Intake, Repo Setup & AI Contract
D1	29 Jun 2026	Project Proposal, Feasibility & Business Considerations
D2	3 Jul 2026	Software Requirements Specification
D3	10 Jul 2026	Product Backlog, UX Prototype & Sprint 1 Plan
D4	17 Jul 2026	Architecture, CI/CD Baseline & Mid-Sem Defense
D5	24 Jul 2026	Sprint 1 Review & Working Prototype v0.1
D6	31 Jul 2026	Sprint 2 Review, Test Plan & Quality Evidence
D7	7 Aug 2026	Sprint 3 Review, Security/Privacy/AI Audit & Observability
D8	14 Aug 2026	Sprint 4 Review, Release Candidate & User Manual Draft
D9	21 Aug 2026	Final Demo, User Manual, Ops Guide & Client Handoff
D10	28 Aug 2026	Final Repo Freeze, Post-Mortem, Peer Evaluation

9. Repository & Governance

- Team repository: github.com/RachelNhyira23/DCIT-208-GROUP-25
- All work tracked via GitHub Issues, feature branches, and reviewed Pull Requests
- AI usage logged transparently in AI_USAGE.md for every deliverable
- Weekly meetings: Tuesdays 3–6pm and Fridays 6–8pm via Google Meet
- Internal submission freeze: 3:00pm, two hours ahead of the official 5:00pm Sakai deadline